

Docker Fundamentals

Cognizant Digital Nurture 5.0 – Deep Skilling Program

Week 7 – DevOps & Cloud

Prepared by: Ashreen Fathima A

Department: Computer Science and Design

Institution: RMK Engineering College

Email: ashreenfathima26@gmail.com

Table of Contents

1. Introduction
 2. What is Docker?
 3. Why Docker?
 4. Problems Before Docker
 5. Containers vs Virtual Machines
 6. Docker Architecture
 7. Docker Components
 8. Docker Images
 9. Docker Containers
 10. Dockerfile
 11. Docker Hub
 12. Docker Lifecycle
 13. Common Docker Commands
 14. Advantages of Docker
 15. Best Practices
 16. Conclusion
-

1. Introduction

Modern software applications must be portable, scalable, and easy to deploy across different environments. Traditionally, developers often encountered the problem of applications working correctly on one machine but failing on another due to differences in operating systems, software versions, or dependencies.

Docker was introduced to solve this problem by providing containerization technology, allowing applications and their dependencies to be packaged together into lightweight, portable containers.

Today, Docker is one of the most widely used tools in DevOps, Cloud Computing, and Microservices-based architectures.

2. What is Docker?

Docker is an open-source containerization platform that enables developers to package an application together with all its required libraries, dependencies, configuration files, and runtime environment into a standardized unit called a **container**.

A Docker container runs consistently across different systems regardless of the underlying operating system.

Definition

Docker is a platform for developing, shipping, and running applications using lightweight containers.

3. Why Docker?

Docker addresses several common software deployment challenges.

Before Docker

- Application works on developer's machine only.
- Missing libraries on production servers.
- Different operating systems.
- Version incompatibilities.
- Slow deployment.
- Difficult maintenance.

After Docker

- Consistent environment.
 - Faster deployment.
 - Easy scaling.
 - Improved portability.
 - Better resource utilization.
-

4. Problems Before Docker

Traditional software deployment often suffered from:

- Dependency conflicts
- Environment mismatch

- Manual installation
- Configuration errors
- Long deployment cycles
- High infrastructure cost

Docker eliminates these issues by packaging everything required to run an application into a single container.

5. Containers vs Virtual Machines

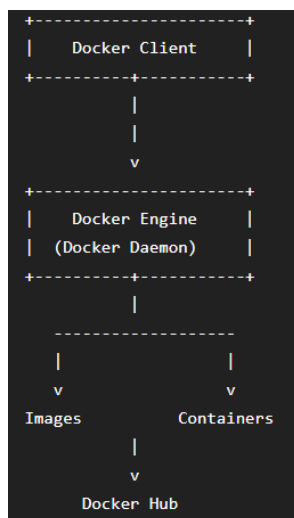
Feature	Docker Containers	Virtual Machines
Startup Time	Seconds	Minutes
Size	Lightweight	Large
Performance	High	Moderate
OS Required	Shares Host Kernel	Separate Guest OS
Resource Usage	Low	High
Boot Time	Very Fast	Slow

Key Difference

Containers share the host operating system kernel, while Virtual Machines require a complete guest operating system.

6. Docker Architecture

Docker follows a client-server architecture.



Components

- Docker Client
 - Docker Engine
 - Docker Daemon
 - Docker Images
 - Docker Containers
 - Docker Registry (Docker Hub)
-

7. Docker Components

Docker Client

The Docker Client allows users to execute Docker commands.

Example:

`docker build`

`docker run`

`docker pull`

`docker push`

Docker Daemon

The Docker Daemon runs in the background and manages containers, images, networks, and storage.

Docker Registry

A Docker Registry stores Docker Images.

Examples:

- Docker Hub
 - Azure Container Registry
 - Amazon Elastic Container Registry
 - Google Container Registry
-

8. Docker Images

A Docker Image is a read-only template used to create Docker Containers.

Characteristics:

- Immutable

- Portable
- Reusable
- Version Controlled

Example:

ubuntu

mysql

nginx

openjdk:21

9. Docker Containers

A Docker Container is a running instance of a Docker Image.

Each container has:

- Application
- Runtime
- Libraries
- Configuration
- Dependencies

Containers are isolated from one another while sharing the host operating system.

10. Dockerfile

A Dockerfile is a text file containing instructions used to automatically build Docker Images.

Example:

FROM openjdk:21

WORKDIR /app

COPY app.jar app.jar

EXPOSE 8080

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Common Dockerfile Instructions

Instruction	Purpose
FROM	Base Image
COPY	Copy Files
WORKDIR	Set Working Directory
RUN	Execute Commands
EXPOSE	Open Port
CMD	Default Command
ENTRYPOINT	Main Application

11. Docker Hub

Docker Hub is the official public repository for Docker Images.

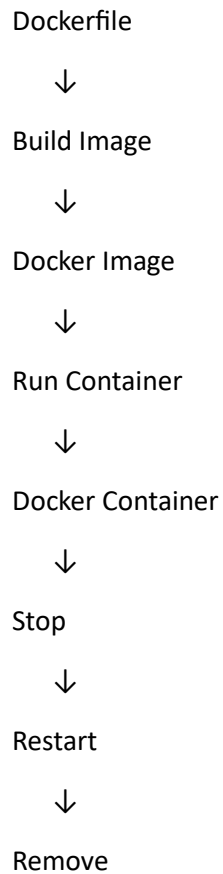
Developers can:

- Download Images
- Upload Images
- Share Images
- Maintain Version History

Popular Images

- Ubuntu
 - Nginx
 - MySQL
 - PostgreSQL
 - Redis
 - MongoDB
 - OpenJDK
-

12. Docker Lifecycle



13. Common Docker Commands

Command	Description
<code>docker --version</code>	Check Docker Version
<code>docker images</code>	List Images
<code>docker ps</code>	Running Containers
<code>docker ps -a</code>	All Containers
<code>docker pull image</code>	Download Image
<code>docker run image</code>	Run Container
<code>docker stop container</code>	Stop Container
<code>docker start container</code>	Start Container
<code>docker rm container</code>	Delete Container
<code>docker rmi image</code>	Delete Image
<code>docker build -t app .</code>	Build Image

Command	Description
---------	-------------

<code>docker logs container</code>	View Logs
------------------------------------	-----------

14. Advantages of Docker

- Lightweight
 - Fast Deployment
 - Platform Independent
 - Easy Scaling
 - High Performance
 - Efficient Resource Utilization
 - Better Collaboration
 - Supports Microservices
 - Simplifies CI/CD
 - Faster Software Delivery
-

15. Best Practices

- Use official base images.
 - Keep images lightweight.
 - Avoid running containers as root.
 - Remove unnecessary files.
 - Use `.dockerignore`.
 - Tag image versions appropriately.
 - Store secrets securely.
 - Regularly update base images.
-

17. Conclusion

Docker has transformed modern software development by enabling consistent, portable, and efficient application deployment through containerization. It plays a crucial role in DevOps practices, Continuous Integration/Continuous Deployment (CI/CD), cloud-native applications, and microservices architecture. Understanding Docker fundamentals provides a strong foundation for learning advanced DevOps tools and cloud platforms.